

Hardware-Conscious Performance Engineering for Distribution-Aware Dataset Search

Thesis status — May 2026

Tarik Abu Mukh
Advisor: Lennart Behme

TU Berlin, DIMA group

14 May 2026

What this talk is about

- The Rust port works. Hundreds-of-times faster than Python.
- But that's not the contribution.
- The thesis says three things:
 - ① There are **four reasons** this workload slows down on Sapphire Rapids.
 - ② Stacking optimisations **makes things worse** in a structured way — five times, on five different bottlenecks.
 - ③ Together, this forces a **dispatch policy** that picks the right build per situation.
- The speedup is just proof the engineering worked.

The proposal vs. what was delivered

Original proposal: 6–20× speedup over Python in Rust.

Current motto

“I don’t really care about the speedup — what I care about is how we break the bottlenecks.”

So the thesis has two sides:

- **Speedup:** comfortably exceeded.
- **Mechanism:** the real argument.

What	Speedup
Search vs. Python	400–880×
From Rust alone	~150×
From ceiling-aware	3–6× extra
End-to-end (user-visible)	100–200×
Index building (rebinning)	~18×
Correctness	precision = recall = 1.000

The hardware: Sapphire Rapids 8468H

- 2 sockets, 96 physical cores total (192 logical with hyperthreading)
- ~ 600 GB/s DDR5 bandwidth aggregate (~ 300 GB/s per socket)
- 105 MiB L3 cache per socket
- Cross-socket interconnect (UPI) is **about $2\times$ slower** than on-socket access
- AVX-512 with FP16 support

Every bottleneck the thesis identifies is grounded in this specific hardware.

Four reasons **this workload** slows down

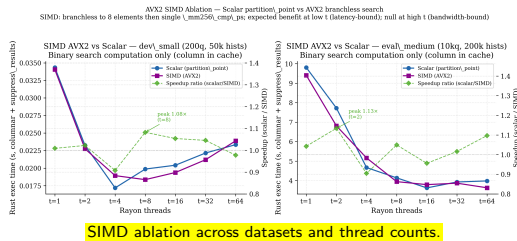
Each one binds under a different thread count.

#	Bottleneck	Binds at	Best attack
(i)	Single-thread compute / L1 latency	$t \leq 8$	— (does not bind in practice)
(ii)	Shared L3 cache fills up	$t \approx 16$	Shrink data (f16)
(iii)	On-socket DRAM bandwidth	$t \in [32, 48]$	packed-ids, pooled, mimalloc
(iv)	Cross-socket interconnect (UPI)	$t > 48$	Pin to one socket (numactl)

Each of the next four slides shows the evidence for one ceiling.

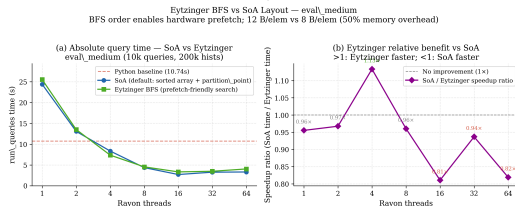
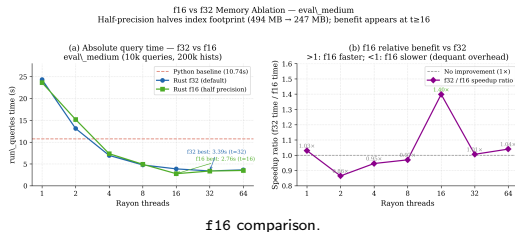
Ceiling (i) — compute / L1: doesn't bind

- Naïve intuition: “vectorise the inner loop, get more done per cycle.”
- Reality: every latency-end ablation measures within $\pm 7\%$.
 - AVX2 SIMD: $\leq 1.13\times$
 - 8-way batch search: -17% at $t = 16$
 - 4-way k-ary: $+1.19\times$ at $t = 16$, but the gain is from *fewer memory misses*, not faster compute
 - Horizontal AVX-512: $+14\%$ at $t = 64$ only
- This ceiling is named for completeness. The workload isn't actually compute-bound here.



Ceiling (ii) — shared L3: the sharp pair

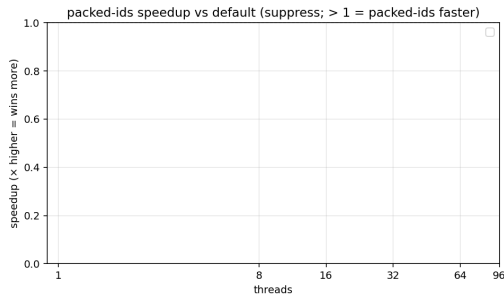
- At $t \approx 16$, threads compete for the 105 MiB shared L3.
- Two optimisations differ *only* in which direction they move the index footprint:
 - f16 halves the values $\rightarrow$ **+40% win at $t = 16$**
 - eytzingier adds $\sim 50\%$ padding $\rightarrow$ **-19% loss at $t = 16$**
- Same direction, opposite results.
- This single experiment isolates **L3 capacity as the binding constraint** better than anything else in the thesis.



Eytzingier comparison (mirror image).

Ceiling (iii) — on-socket DRAM bandwidth

- At $t \in [32, 48]$, threads on one socket saturate DDR5 bandwidth.
- Optimisations that *reduce memory traffic* win:
 - packed-ids: bit-pack histogram IDs, —37% emit bandwidth $\rightarrow$ wins at $t = 64$ on big data
 - pooled: pre-allocated per-query output buffer, biggest single win at $t = 1$ ($1.81\times$)
 - mimalloc: replace glibc allocator, $1.33\times$ at $t = 16$
 - pin-cores: physical-core pinning, +23% at $t = 32$
- Compute-targeting optimisations on this regime: null.



packed-ids speedup across thread counts.

Ceiling (iv) — cross-socket UPI

- Above $t = 48$, the workload spans both NUMA sockets.
- Pages fetched from the wrong socket cost roughly $2\times$ longer.
- Four-configuration numactl probe at $t = 32$ on *c256_56gb*:

Configuration	Wall (s)	Delta
Unpinned (first-touch, default Linux)	17.96	baseline
<code>numactl --cpunodebind=0 --membind=0</code>	15.55	-12.4% win
<code>numactl --interleave=0,1</code>	25.14	+40.0% loss
<code>numactl --membind=0 --physcpubind=48-79</code>	19.08	+1% (worst case)

- Single-socket placement is a deployment flag — zero code change.
- Naïve “balance across both sockets” is the worst choice.
- This ceiling was hidden inside (iii) for the whole campaign — the probe split them apart.

The stacking pattern — five times, five bottlenecks

Naïve intuition: “if A makes it faster and B makes it faster, A+B should be faster still.”

Reality: if A already addressed the binding bottleneck, B has nothing left to fix and becomes pure overhead.

§	Composition (baseline ← extra)	Already-attacked bottleneck	Outcome
3.7.6	bestofsuite ← packed-ids	DRAM bw (f16, pooled)	Mixed NEG
3.9.7	bestofsuite ← query-batch	DRAM bw (cold-load halved by f16)	5/5 NEG, +16 to +44%
3.10	qbatch K=128 ← morsel	L3 (qbatch already amortised)	+8 to +12% at c256
3.11	packed-ids ← local-ids	DRAM bw (packed already saved)	+170 to +243%
3.12	first-touch ← numactl --interleave	UPI (first-touch keeps clusters local)	+26 to +41%

- Five *different* ceilings → pattern is structural, not coincidence.
- Each instance has **perf-counter evidence** for the mechanism.
- Falsifiable: a positive composition against an un-attacked ceiling would refute the claim.

What this means: pre-flight ceiling identification

The methodological take-away:

Before adding any optimisation:

- 1 Measure **which ceiling is binding right now** — not at the bare default, but at the current composition.
- 2 If a coarser optimisation already attacks that ceiling, the finer one will fail.
- 3 Predict the saving via bandwidth-budget arithmetic — but treat the prediction as falsifiable.

All five negative-composability instances had a clean a-priori case for positive composition. Every one was falsified at scale.

The rule generalises beyond this codebase.

Traditional ablation methodology — “implement, measure, accept if positive” — isn’t sufficient on hardware with multiple stacked ceilings.

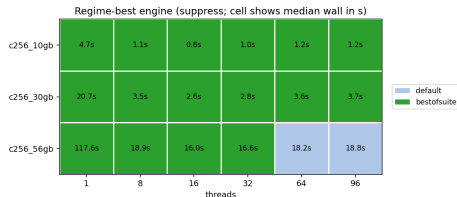
The operational consequence: dispatch-on-regime

- No single build wins everywhere.
- Different ceilings bind at different thread counts.
- Solution: pick the build per regime.

`finder/execution/dispatch.py`: looks at $(n_{\text{clusters}}, n_{\text{hist}}, n_{\text{threads}}, \text{placement})$ and returns the best build for that cell.

Validation on three datasets:

- **17/18 cells exactly right**
- **18/18 within 5%** of regime-best



Dispatch policy's pick vs. measured regime-best.

Out-of-distribution test: does the picker generalise?

Built a fourth dataset: c1024_56gb

- Same histograms file, same queries, same bin density
- K-means at K=1024 collapsed to **610 effective clusters**
- → **3.2× sparser clusters** than c256_56gb's 191

Without changing the dispatch code:

- **6/6 cells within 5%** of regime-best
- **All 6 within 1%**
- Two cells weren't *exactly* right but both shifted toward bestofsuite — mechanism-consistent with sparser clusters

Combined validation record: **23/24 cells within 5%**

- 17/18 in-distribution
- 6/6 out-of-distribution
- Spans a 3.2× density range

Did the structure reproduce at the new dataset?

Phase 4: tested all 8 structural claims at c1024_56gb.

#	Claim	Verdict
Ceiling (i)	simd stays null	Reproduces
Ceiling (ii) f16	f16 sharp pair	Refined — footprint-dependent
Ceiling (ii) aos	aos negative control	Reproduces
Ceiling (iv) ssoc	Single-socket beats unpinned	Reproduces (−12%)
Ceiling (iv) intl	Interleave regresses	Reproduces (+34 to +40%)
§3.7.6	Mixed NEG	Consistent with c256
§3.9.7	5/5 NEG, even larger	Reproduces, amplified
§3.10	c256's +8 to +12% NEG	Attenuates to within-noise
§3.11	Catastrophic NEG	Reproduces
§3.12	Interleave NEG	Reproduces

5/8 strong + 2/8 clean nulls + 2/8 refinements + 0/8 falsifications.

Mechanism shift on the new dataset

The two “refinements” aren’t failures — they’re *informative*:

f16 alone sign-flips at c1024

- c256: f16 won at $t = 16$ because per-cluster array (105 KB) exceeded L1
- c1024: per-cluster array is ~ 33 KB — already fits L1 at f32
- So halving has no headroom to recover, and the conversion cost shows
- **But bestofsuite bundle still wins 18%** at both densities — f16’s contribution is *compositional*, not standalone

§3.10 morsel: reliable NEG attenuates to within-noise

- c256: +8 to +12% regression at every cell — tight margins
- c1024: all 5-rep 95% CIs straddle zero
- Sparser clusters mean smaller cluster cold-loads, less for the L3 to absorb → less work for morsel to fail at
- Direction-consistent with the mechanism story; just below the measurement floor

Honest variance review

First Phase 4 draft over-claimed in two places. Cowork flagged them. Both were fixed before this presentation:

- **Original claim:** “§3.10 morsel reproduces cell-dependently at c1024.”
- **Variance check:** 5-rep 95% CIs all straddle zero. Revised to: “c256’s reliable NEG attenuates to within-noise at c1024.”
- **Original observation:** “simd wins -6.7% at c1024 $t = 64$ — unexplained.”
- **Variance check:** $-3.88\% \pm 7.03\%$. Straddles zero. Confirmed clean null. No anomaly.

Plus one extra ablation: ran bestofsuite minus f16 at c1024 $t = 16$ (5 reps).

- Removing f16 costs $\sim 11\%$ point-estimate vs. the full bundle
- f16 contributes *positively* inside the bundle, even though standalone it loses
- Compositional positive contribution — the bundle is more than the sum of its parts

What's still to do

Thesis writeup:

- Chapter 6 (Related Work) — reframe against composability literature + NUMA-aware OLAP
- Chapter 7 (Conclusion / Future Work) — reconcile with G1–G5 framing
- Bibliography — a few new cites (Polychroniou, Leis, Bandle, Graham, Ferragina)

Stretch experiments documented as future work, each with a pre-registered prediction and a falsifier:

- **F1:** Transparent Huge Pages probe — either hardens §3.11 or surfaces a 5th ceiling (page-walk pressure). ~ 1 day.
- **F2:** NUMA-aware build for $t > 48$ — shard clusters across sockets. Either attacks ceiling (iv) or becomes a 6th neg-comp instance. $\sim 3\text{--}5$ days.
- **F3:** Density-axis dispatch refinement at c512_56gb and c4096_56gb — locates the histograms-per-cluster boundary. $\sim 1\text{--}2$ days, zero risk.

Closing

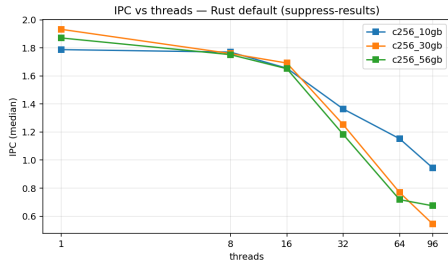
- Speedup: comfortably exceeded the $6\text{--}20\times$ target.
- But the contribution is the **mechanism**, not the speedup.
- Four ceilings, identified with perf-counter evidence.
- Five-instance negative-composability pattern, cross-axis + cross-density.
- Pre-flight ceiling-identification as the methodological recommendation.
- Dispatch-on-regime: 23/24 cells within 5% across a $3.2\times$ density range.
- Zero outright falsifications across both the original and OOD campaign.

Questions?

Backup slides

Backup: “Why is IPC so low? Why not optimise it?”

- Low IPC at high t is the **symptom of being memory-bound**, not bad code.
- Cycles aren't wasted — they're waiting on memory fetches.
- Raising IPC means *reducing memory waits*, which is what every working optimisation does.
- Best single counterexample: `local-ids` (§3.11) drops IPC from 2.6 to 1.1 *and* runs slower.
- IPC is a *diagnostic*, not a target.



IPC drops as t increases — the signature of bandwidth-bound execution.

Backup: “Why doesn't more inter-query parallelism help?”

Tried all four granularities:

Granularity	Mechanism	Result
Coarse (Rayon)	Distribute 10K queries across cores	Headline 6–7× at $t = 16$
Mid (qbatch)	K queries amortise each cluster cold-load	Regime-dependent positive
Fine SIMD	16 queries lock-step through bisection	+14% at $t = 64$, lane divergence
Cross-thread	Cooperative caching (morse1)	+8 to +12% regression

- The naïve intuition assumes the ceiling is *compute*. It isn't.
- Adding more in-flight queries increases L3 pressure, bandwidth contention, and UPI traffic — not headroom.
- §3.9.7 is the cleanest evidence: 5/5 cells *worse* with more parallelism, and the regression is *larger* at sparser clusters.

Backup: “Is the stacking pattern just folklore?”

No — folklore is general intuition. The thesis claim is more specific:

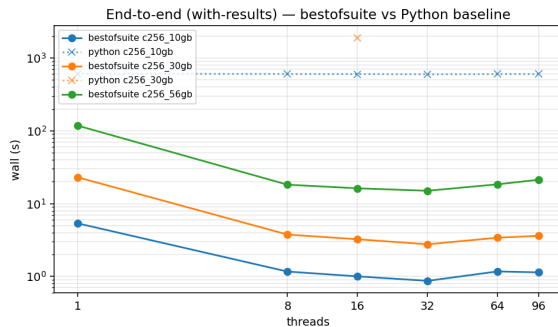
- **5 instances on 5 different ceilings** — not just one stacked-pair failed
- Each has **perf-counter evidence** for *why* the binding ceiling was already collected
- Cross-density-robust: the strong instances reproduce at the OOD dataset
- **Falsifiable**: a counterexample would be an optimisation that succeeds against a ceiling the baseline does *not* address. None of the 5 instances satisfy that counterfactual.

Positioning vs. existing literature:

- Selinger-era cost-based optimisation assumes additive operator costs — §3.13 refines this at the hardware-ceiling level
- Caladan / heartbeat scheduling: orthogonal resource axes — different problem
- Brooks / Wulf-McKee “no silver bullet” / “memory wall”: anticipates rhetorically; this thesis identifies the specific structural shape

Backup: end-to-end vs. Python

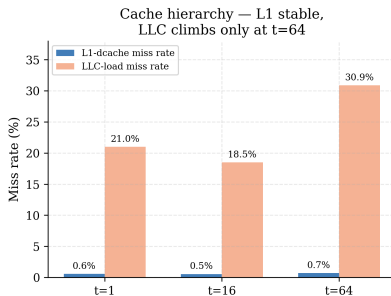
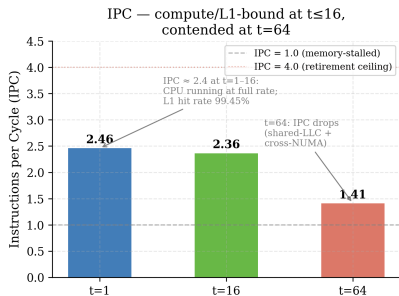
- Search-phase speedup (suppress mode, isolates the index):
 - c256_10gb: $\sim 400\times$
 - c256_30gb: $\sim 880\times$
 - c256_56gb: $\sim 600\times$
- End-to-end (with Python boxing back into result objects): $\sim 100\text{--}200\times$.
- Rebinning kernel: $\sim 18\times$ on a separate axis.
- All preserve precision = recall = 1.000.



End-to-end bestofsuite speedup across the three datasets.

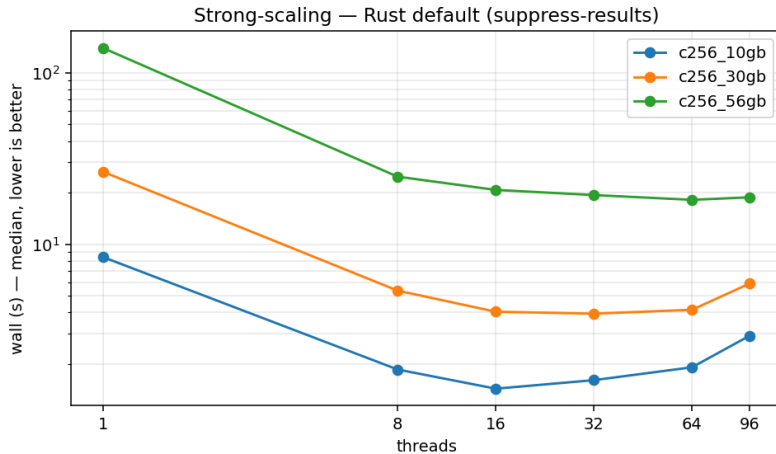
Backup: Roofline

Hardware performance counters — eval_medium (perf stat --delay=40000, 5×10k queries)
IPC 2.46 at t=1 (L1-bound, branchless CMOV); drops to 1.41 at t=64 (memory-subsystem-contended)



Roofline: where this workload sits relative to compute and bandwidth ceilings.

Backup: scaling across thread counts



Default-build strong scaling. The knee at $t = 16$ on c256_56gb is where ceiling (ii) starts to bind.